

Task Category	Today	With a Better Future AI
Just Me	Making career decisions (choosing internships, jobs, or Master's options). Requires personal goals and values.	Still a Just Me task. AI can advise, but the final decision should remain mine.
Centaur Task	Learning Python, React, Django, and AI concepts using explanations and examples. AI accelerates learning, but I must understand and practice myself.	AI becomes a real-time tutor that adapts lessons, quizzes, and projects to my progress.
Centaur Task	Building my FYP and coding assignments with AI suggestions while I write and debug code. Requires back-and-forth collaboration.	AI handles more implementation while I focus on architecture and validation.
Delegated Task	Resume, cover letter, and LinkedIn improvements. AI drafts content, but I verify accuracy and personalize it.	AI automatically tailors applications for each job with minimal editing.
Delegated Task	Interview preparation and mock technical questions. AI generates practice, but I must evaluate my own performance.	AI simulates realistic interviewers and provides detailed personalized coaching.
Delegated Task	Summarizing lectures, research papers, and documentation. AI saves reading time, but I verify important details.	AI creates accurate summaries, flashcards, and personalized study plans automatically.
Collaborate Task	Debugging Python or JavaScript code. AI suggests fixes, while I test and understand them.	AI identifies root causes, proposes fixes, and explains them before I approve changes.
Delegated Task	Creating reports, documentation, and presentations. AI generates drafts, but formatting and accuracy need review.	AI produces polished documents with minimal manual editing.
Delegated Task	Email drafting and professional replies. AI writes the first draft, but I review before sending.	AI handles routine professional emails automatically, asking only when decisions are required.
Collaborate Task	Brainstorming project ideas and software designs. Best results come from combining AI suggestions with my creativity.	AI acts as a design partner, generating prototypes and evaluating trade-offs instantly.
Fully Automate	Weekly task tracking and productivity reports. AI can collect progress data automatically.	AI continuously monitors work, updates progress, and suggests schedule improvements.
Just Me	Building communication skills, teamwork, and professional relationships. Genuine human interaction cannot be delegated to AI.	AI can coach and provide feedback, but real conversations and relationships remain my responsibility.

Claude

New chat

Chats

Projects

Artifacts

Code

Customize

Products

Design

Recents

Try the basics

0 / 3

Bring history from another AI

So you're not starting from scratch

Connect your everyday tools

Claude gives better answers when it understands what matters to you

Get the desktop app

Hand off tasks, code with Claude, and do more from your desktop

AH

Ayesha Hassan

Free plan

Preparing for software engineering roles

Share

Project name: Career & Software Engineering Assistant

I am a recent Computer Science graduate preparing for software engineering roles. Help me improve my Python, backend development, and problem-solving skills. Be concise, explain your reasoning clearly, and point out mistakes or better approaches when reviewing my work. Prioritize accuracy and practical guidance.

Recognized user's software engineering development goals

Got it — I'll act as a technical mentor: concise feedback, clear reasoning, and I'll flag mistakes or suggest better approaches rather than just validating what you have.

To make this useful right away, a couple of quick questions:

What would help most right now?

Review code you've already written (paste it and I'll critique)

Work through a specific problem/concept together (e.g., a LeetCode-style question, a system design topic, a Python gotcha)

Build something backend-related step by step (e.g., a REST API, a small service) with me checking your work as you go

General prep — I suggest a structured plan (Python fundamentals → backend concepts → problem-solving practice)

Where are you currently at?

Comfortable with Python syntax but shaky on idiomatic patterns / performance

Know Python well but haven't built real backend systems (APIs, databases, auth, etc.)

Solid on both, mainly need problem-solving/interview practice

Starting fresh on all three

Tell me which applies (or just dive in with actual code/questions) and I'll get specific.

starting fresh on all three

Architected practical learning progression with immediate actionable steps

Good — starting fresh means we build a foundation that doesn't need unlearning later. Here's the reasoning behind how I'll structure this, then a concrete starting point.

Why this order

1. Python fundamentals with an eye toward idioms, not just syntax — writing *Pythonic* code (list comprehensions, context managers, proper error handling) is what separates "knows Python" from "codes in Python."

2. Problem-solving in parallel, not after — data structures/algorithms practice should start immediately, even while Python is still shaky, because the thinking skill (breaking down problems, reasoning about complexity) takes longer to build than syntax does.

3. Backend concepts once Python is stable — REST APIs, databases, auth build on solid Python; jumping in too early means debugging language issues instead of learning concepts.

Proposed 3-track plan (roughly parallel, not strictly sequential)

Track 1 — Python depth (~2-3 weeks)

Core: functions, OOP; error handling, file I/O

Idiomatic Python: comprehensions, generators, decorators, context managers (`with`)

Standard library essentials: `collections`, `itertools`, `typing`.

Track 2 — Problem-solving (ongoing, daily-ish)

Big-O reasoning from day one — every solution we discuss, I'll ask "what's the time/space complexity, and why?"

Core structures: arrays, strings, hash maps, linked lists → stacks/queues → trees/graphs → DP

We do problems together: you attempt, I review and point out inefficiencies or bugs, not just "correct/incorrect"

Track 3 — Backend fundamentals (starts once Track 1 is stable)

HTTP basics, REST principles

Build a small API (I'd recommend **FastAPI** — modern, type-hint driven, teaches good habits)

Databases: SQL basics, then an ORM (SQLAlchemy)

Auth, testing, and eventually deployment basics

Let's set expectations before we start:

🌟

Write a message...

+

Sonnet 5 Medium

Claude is AI and can make mistakes. Please double-check responses.



CERTIFICATE of COMPLETION

Ayesha Hassan

has completed

AI Fluency: Framework & Foundations

ANTHROPIC

